

Plan Maestro de Mejora - RHDreamsApp 2026

Fecha: 2026-06-02

Repositorio: RHDREAMSAPP2026

Objetivo: mejorar la lgica de chats, flujos, arquitectura, frontend, backend y UX/UI de la aplicacin.

1. Resumen ejecutivo

La aplicacin ya tiene una base funcional fuerte: React + Vite, Firebase/Firestore, backend Express, rutas serverless para Vercel/Cloudflare, mdulos de candidatos, mensajes, agentes IA, flujos IA, reportes, Google Workspace e integraciones.

La prioridad debe ser convertirla de una app demostrativa o semiprototipo en una plataforma de reclutamiento robusta, segura y escalable. Los puntos ms urgentes son: cerrar reglas de Firestore, mover la lgica crtica del frontend al backend, formalizar conversaciones y mensajes, usar historial real en IA, unificar automatizaciones con workflows, y definir una arquitectura de despliegue nica.

2. Diagnostico principal

2.1 Seguridad

- Las reglas de Firestore deben dejar de permitir lectura y escritura pblica.
- Las operaciones sensibles deben requerir usuario autenticado, rol y empresa.
- Los endpoints de IA deben validar identidad real, no auth opcional.
- Deben eliminarse secretos fallback o valores por defecto inseguros.
- El frontend no debe escribir directamente mensajes, automatizaciones, agentes o etapas crticas sin pasar por backend.

2.2 Chats

- La pantalla de mensajes infiere conversaciones desde candidatos y mensajes, pero falta una entidad formal Conversation.
- La respuesta IA se guarda como WhatsApp aunque el canal original pueda ser Instagram, Messenger, email o manual.
- La IA recibe historial desde el cliente, pero el backend debe convertirlo en contexto real para el modelo.
- Las etapas no deben cambiar por palabras clave en el texto de la IA; deben cambiar por intencin estructurada y validada.
- Falta trazabilidad de estado: enviado, entregado, leído, fallido, pendiente de aprobacin, escalado humano y borrador IA.

2.3 Flujos y automatizaciones

- Hoy conviven automatizaciones simples en el hook de datos y un builder de workflows ms avanzado.
- El workflow actual valida variables y marca pasos como completados, pero no ejecuta acciones reales de negocio.
- Hace falta una sola fuente de verdad para triggers, condiciones, pasos, aprobaciones, logs y resultados.
- Las aprobaciones humanas deben integrarse al chat y no quedar separadas en una pantalla de flujos.

2.4 Backend

- Hay Express, Vercel serverless y Cloudflare Pages Functions con comportamientos distintos.
- La app necesita decidir una arquitectura oficial: backend Node persistente o serverless con proveedores compatibles.

- WhatsApp normal con Baileys requiere servidor persistente; si se quiere serverless, conviene WhatsApp Business Cloud API.
- La lgica de negocio debe vivir en servicios backend: conversaciones, mensajes, IA, flujos, estados de candidato e integraciones.

2.5 Frontend y UX/UI

- La navegacin es completa y est bien segmentada por mdulos.
- La pantalla de chat necesita estados operativos claros: IA pensando, requiere aprobacin, escalado, error, no enviado, SLA vencido.
- Los estados vacos deben invitar a crear candidato, importar conversacin, conectar canal o simular una conversacin.
- El usuario debe poder ver por qu la IA tom una decisin, qu datos us y qu workflow se ejecut.

3. Modelo de datos propuesto

3.1 Conversation

```
Conversation {
  id: string;
  companyId: string;
  candidateId: string;
  channel: "whatsapp" | "instagram" | "messenger" | "email" | "manual";
  provider: "baileys" | "meta" | "gmail" | "manual";
  status: "open" | "pending_ai" | "pending_human" | "closed";
  assignedAgentId?: string;
  assignedRecruiterId?: string;
  lastMessageAt: number;
  unreadCount: number;
  tags: string[];
  aiMode: "off" | "suggest" | "auto_with_approval" | "auto_send";
}
```

3.2 Message

```
Message {
  id: string;
  companyId: string;
  conversationId: string;
  candidateId: string;
  channel: string;
  direction: "inbound" | "outbound";
  body: string;
  status: "draft" | "pending_approval" | "sent" | "delivered" | "read" | "failed";
  providerMessageId?: string;
  sentAt?: number;
  deliveredAt?: number;
  readAt?: number;
  failedReason?: string;
  aiGenerated?: boolean;
  aiAgentId?: string;
  aiConfidence?: number;
  intent?: string;
  sentiment?: string;
  requiresApproval?: boolean;
  approvedBy?: string;
  approvedAt?: number;
  dedupeKey?: string;
}
```

```
}
```

3.3 WorkflowRun

```
WorkflowRun {  
  id: string;  
  companyId: string;  
  workflowId: string;  
  entityType: "candidate" | "conversation" | "appointment";  
  entityId: string;  
  status: "queued" | "running" | "approval_required" | "completed" | "failed";  
  startedAt: number;  
  finishedAt?: number;  
  logs: WorkflowRunLog[];  
}
```

4. Roadmap recomendado

Fase 0 - Seguridad y control de datos

1. Cerrar Firestore con reglas por request.auth, companyId y rol.
2. Desactivar escrituras pblicas en candidatos, mensajes, agentes, workflows y notificaciones.
3. Validar Firebase ID Token en backend.
4. Sustituir autenticacin opcional por autenticacin obligatoria en endpoints de IA.
5. Agregar auditora mnima: quin cre, actualiz, aprob o envi cada accin.

Fase 1 - Chat robusto

1. Crear entidad Conversation y migrar la pantalla de mensajes para usarla.
2. Agregar estados de mensajes y trazabilidad de proveedor.
3. Corregir el canal de respuesta IA para respetar el canal original.
4. Mostrar estados UX: "IA redactando", "pendiente de aprobacin", "enviado", "fall", "escalado".
5. Agregar panel lateral de candidato dentro del chat con vacante, etapa, tags, ltimos eventos y prximo paso.

Fase 2 - IA conversacional confiable

1. Crear ai-orchestrator.service.ts en backend.
2. Enviar historial real al modelo, no solo el ltimo prompt.
3. Pedir salida JSON estructurada: respuesta, intencin, confianza, siguiente etapa, campos faltantes y escalamiento.
4. Validar la salida con Zod antes de guardar cualquier decisin.
5. Activar aprobacin humana para respuestas sensibles: rechazo, salario, documentos, temas legales o baja confianza.

Fase 3 - Workflows reales

1. Unificar automatizaciones simples y workflows visuales.
2. Crear colecciones workflows, workflowRuns y workflowRunLogs.
3. Convertir pasos simulados en acciones reales: mensaje, IA, condicin, calendario, tarea, webhook y handoff.
4. Agregar simulador de workflow con candidato real.
5. Agregar versionado, publicacin y rollback de flujos.

Fase 4 - Backend productivo

1. Decidir arquitectura oficial.
2. Si se requiere Baileys/WhatsApp normal, usar backend Node persistente en VPS/Render/Fly/Railway.

3. Si se quiere serverless puro, migrar a WhatsApp Business Cloud API.
4. Eliminar divergencia entre Express, Vercel y Cloudflare o documentar responsabilidades claras.
5. Implementar Firebase Admin en backend y repositorios por entidad.

Fase 5 - UX/UI y mtricas

1. Redisear chat con bandejas por estado y prioridad.
2. Crear estados vacos accionables.
3. Integrar aprobaciones IA dentro del chat.
4. Mostrar explicacin de decisiones IA y logs de workflow.
5. Medir KPIs: tiempo de respuesta, show rate, tasa de conversin, escalaciones, tasa de error por proveedor y rendimiento por agente.

5. Arquitectura backend recomendada

La opcin ms prctica para esta app es:

- Frontend en Cloudflare Pages.
- Backend Node persistente para IA, WhatsApp/Baileys, colas y cron jobs.
- Firebase Auth para identidad.
- Firebase Admin en backend para Firestore.
- Firestore como base operacional.
- Cloud Tasks, BullMQ, Redis o cola equivalente para eventos asncronos.
- Workers o funciones serverless solo para endpoints simples si se decide mantener Cloudflare.

Esta arquitectura evita que WhatsApp Web falle por falta de socket persistente y permite ejecutar flujos largos, reintentos, aprobaciones y trabajos programados.

6. Servicios backend propuestos

```
server/services/conversation.service.ts
server/services/message.service.ts
server/services/ai-orchestrator.service.ts
server/services/recruitment-state.service.ts
server/services/workflow-engine.service.ts
server/services/handoff.service.ts
server/services/audit-log.service.ts
server/repositories/candidates.repository.ts
server/repositories/conversations.repository.ts
server/repositories/messages.repository.ts
server/repositories/workflows.repository.ts
server/repositories/appointments.repository.ts
```

7. UX/UI propuesta para chat

- Bandejas: Todos, Sin responder, Pendiente IA, Pendiente aprobacin, Escalados, Cerrados.
- Badges visibles por conversacin.
- Composer con acciones rpidas: sugerir respuesta, aprobar y enviar, agendar, escalar, insertar plantilla.
- Vista de candidato al lado derecho: datos, vacante, etapa, score, prximos pasos, historial y notas.
- Timeline de eventos: mensaje recibido, IA gener borrador, reclutador aprob, workflow ejecutado, cita agendada.
- Filtros por canal, vacante, agente, etapa, prioridad y SLA.

8. Checklist de implementacin inmediata

- [] Cerrar reglas pblicas de Firestore.

- [] Crear Conversation y migrar chat.
- [] Mover triggerAgentDialogue del frontend al backend.
- [] Usar historial real en IA.
- [] Exigir salida JSON estructurada de IA.
- [] Agregar aprobacin humana.
- [] Unificar workflows y automatizaciones.
- [] Definir backend productivo nico.
- [] Crear pruebas unitarias y E2E.
- [] Documentar variables de entorno y despliegue real.

9. Indicadores de xito

- Cero escrituras pblicas en Firestore.
- 100% de respuestas IA auditables.
- Menos falsos cambios de etapa por heursticas.
- Mayor tasa de respuesta en chats.
- Menor tiempo promedio hasta entrevista.
- Mayor show rate de entrevistas.
- Menos errores por proveedor de IA o WhatsApp.
- Workflows con logs y ejecucin reproducible.

10. Prximo paso sugerido

El primer sprint debera enfocarse en seguridad y chat:

1. Bloquear Firestore pblico.
2. Crear Conversation.
3. Crear endpoint backend para mensajes.
4. Migrar auto-respuesta IA al backend.
5. Agregar aprobacin humana antes de enviar respuestas IA.

Con eso la app gana seguridad, control operativo y base slida para mejorar flujos y UX sin seguir acumulando lgica crtica en el frontend.